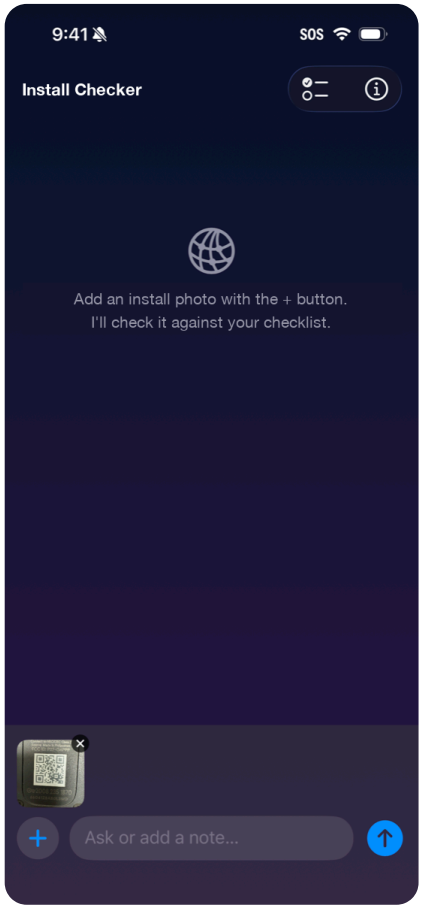
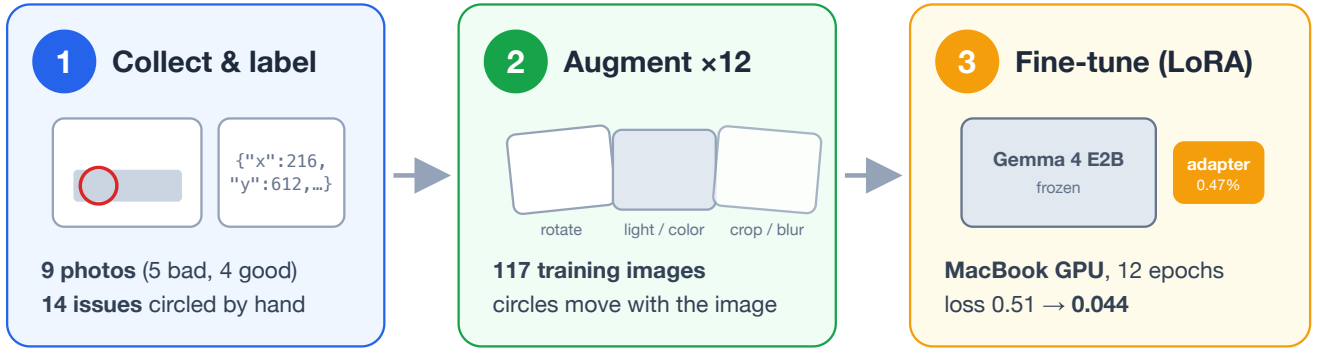
On-Device AI Install Checker — POC

Take a photo of the install → the AI on the phone says PASS / FAIL and circles every problem. No internet needed.



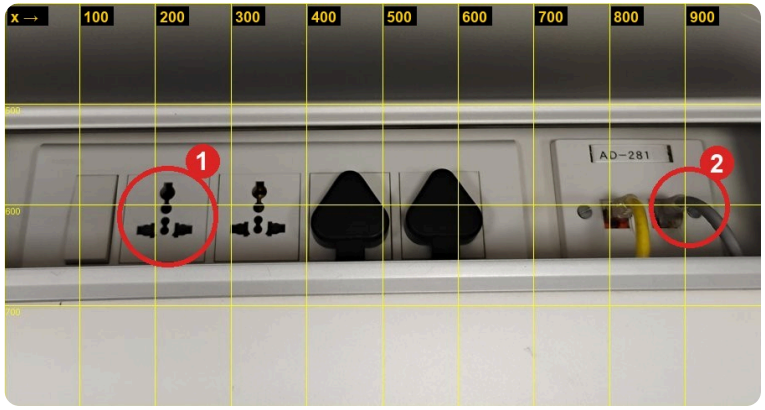
- Model: **Google Gemma 4 E2B**, customized for our install rules
- Runs **fully offline** on an iPhone 15 Pro (Google LiteRT-LM runtime)
- Checks one photo at a time and explains every problem in plain English
- Same model file can run on Android later

1 · How we trained it — Step 1 · 2 · 3



Then: merge adapter → quantize to 8-bit (compress) → one .litertlm file (3.86 GB) for the phone

Example of one label (step 1) — a person circles each problem on the photo and writes what is wrong:



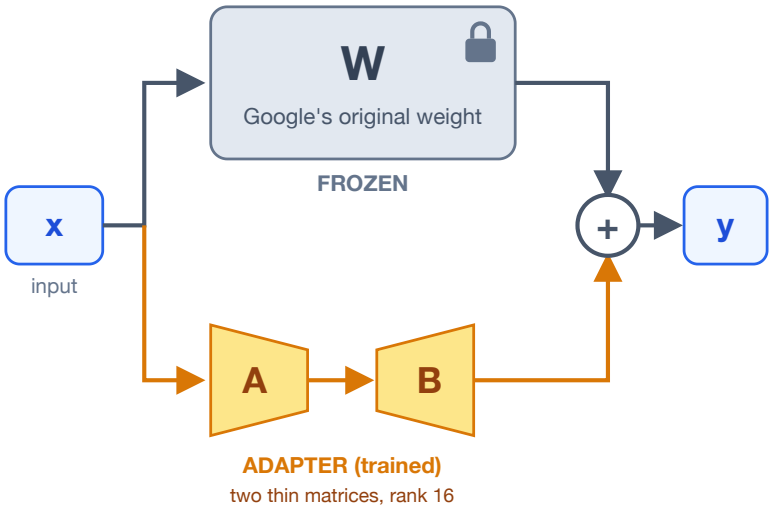

```
{"status": "fail",
 "errors": [
  {"x": 216, "y": 612, "r": 50,
   "desc": "Missing power adapter. Plug the adapter into the first socket."},
  {"x": 905, "y": 603, "r": 40,
   "desc": "Extra grey cable in the right data port. Only the yellow
            LAN cable in the left port is allowed. Unplug the grey cable."}
 ]}
```

x, y = centre of the circle, 0–1000 across the photo width / height (yellow grid) · **r** = circle size · **desc** = what is wrong + how to fix it. This JSON is exactly what the model learns to write.

2 · How LoRA attaches to the base model

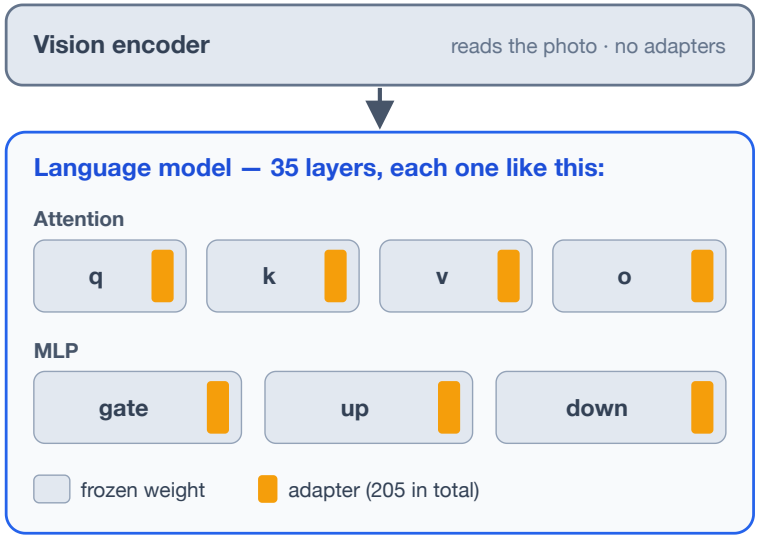
We don't retrain Google's model — we bolt small trainable add-ons onto it, then fuse them in.

1 What an adapter is (one weight)



After training: W′ = W + B·A (adapter fused in)
same size as before → no extra cost on the phone

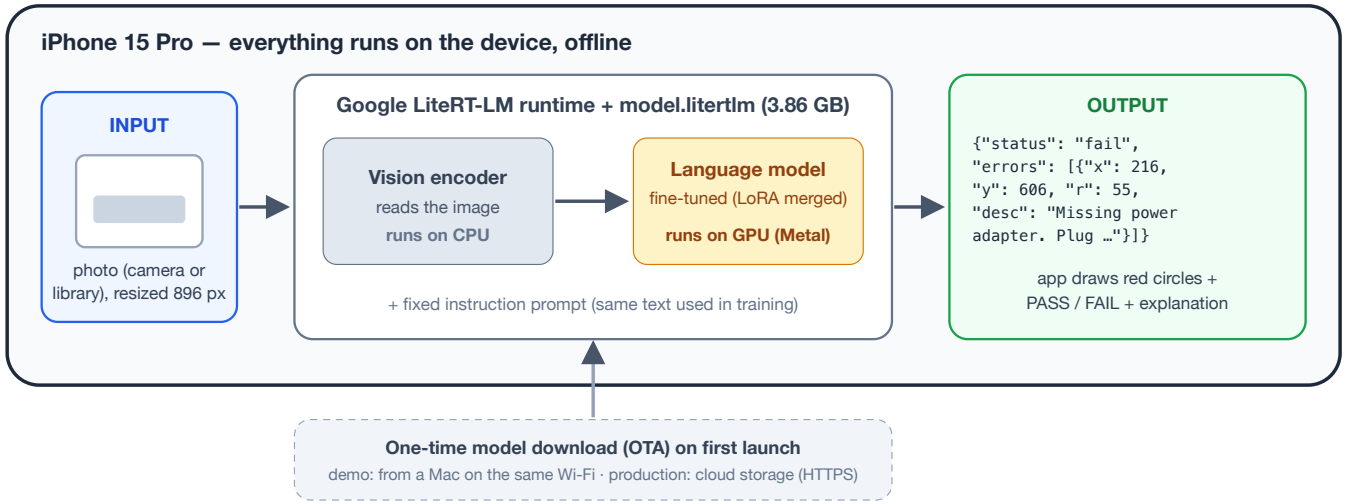
2 Where adapters go in Gemma 4 E2B

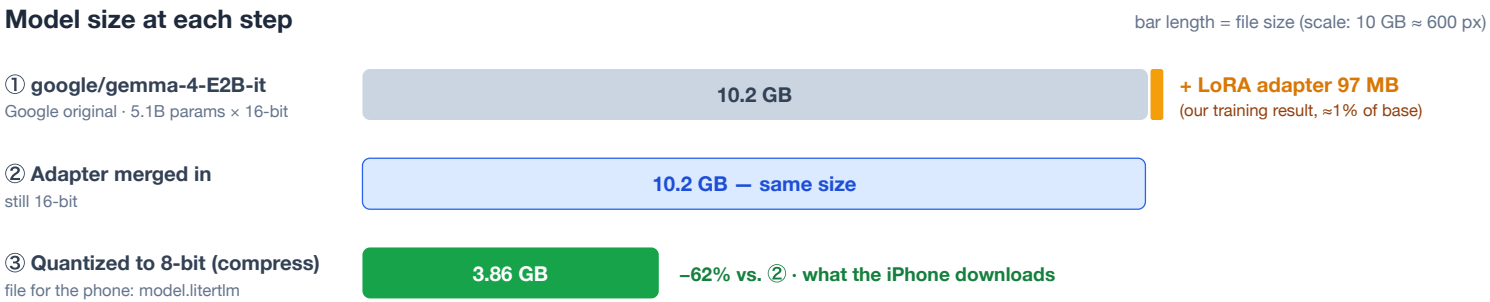


We train only 0.47% of the model
24.2M of 5.1B parameters · adapter file 97 MB

- ① Every original weight **W** stays frozen. Next to it sits a tiny adapter (**A**, **B**); only the adapter is trained. After training it is fused into W, so the phone runs a normal-size model.
- ② Adapters sit in the language part of the model (all 35 layers); the vision part is untouched. We train just **0.47%** of the model.

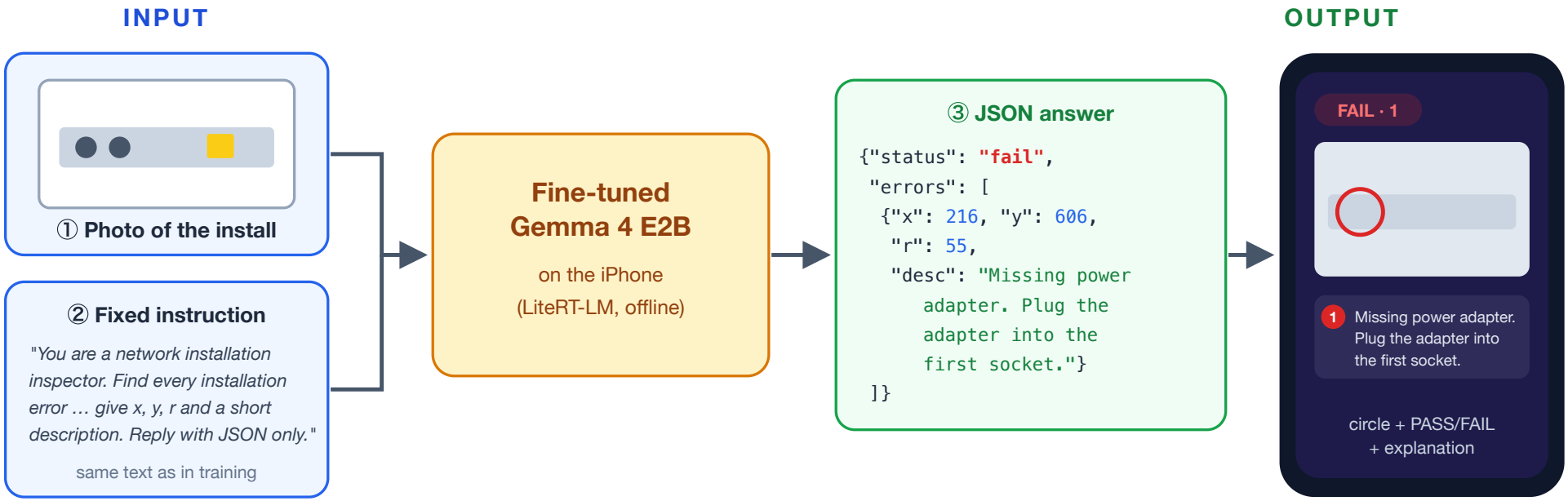
3 · Putting the model into the iOS app





- Base model: Google **Gemma 4 E2B-it** — google/gemma-4-E2B-it on Hugging Face (instruction-tuned, 5.1B parameters stored as 16-bit bf16 = 10.2 GB).
- Base model + our adapter (97 MB) are merged, then **quantized to 8-bit (compressed)** → **one 3.86 GB file**, downloaded once.
- Tech: SwiftUI app · Google **LiteRT-LM** runtime · image part on **CPU**, text part on **GPU (Metal)** · fully offline.

4 · Input → Output





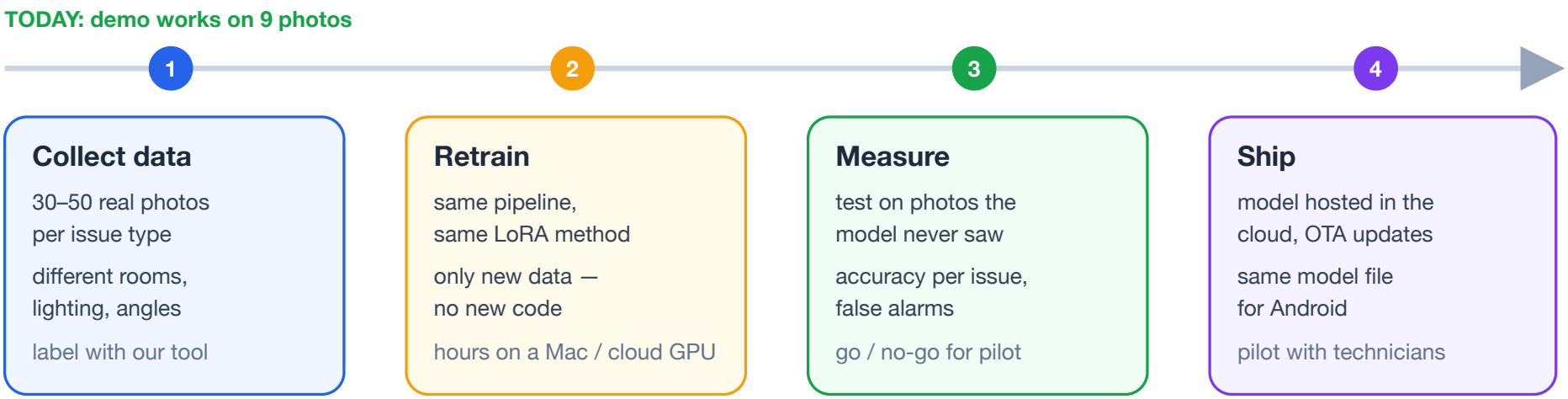
Model output on real photos (same LiteRT-LM runtime as the phone, run on Mac): two failed installs and one correct install.

5 · Numbers

	Result
PASS / FAIL correct	**9 / 9** photos
Model file	**3.86 GB**, downloaded once
RAM used by the app	**~620 MB** (model file is memory-mapped, not copied into RAM)

Honest limits today: only 9 photos, one desk setup — the model has memorized this demo. It will not yet handle new rooms, lighting or equipment. That needs real customer data (next page).

6 · Next steps with customer data



- The pipeline is ready: more photos → same 3 steps → new model file → pushed to phones over the air.
- **Needs from the business:** real install photos from customers/technicians, a paid Apple Developer account (for full memory features), and a cloud location to host the model.